

**IQRA UNIVERSITY, H9 Campus**

Department of Computer Science

**Computer Architecture****Semester Project Report***Ding Dong — Number Guessing Game*

|                        |                                |
|------------------------|--------------------------------|
| <b>Instructor</b>      | Ma'am Tayyaba shehzadi         |
| <b>Semester</b>        | Spring 2026                    |
| <b>Submission Date</b> | June 16, 2026                  |
| <b>Category</b>        | Game-Based Project             |
| <b>Tool</b>            | MASM / Irvine32 (x86 Assembly) |

**Group Members**

| # | Student Name     | Roll Number | Contribution                       |
|---|------------------|-------------|------------------------------------|
| 1 | Muhammad Danish  | 41510       | Lead Developer / Code Architecture |
| 2 | Mohsin Ali       | 41420       | Idea and procedure                 |
| 3 | Abdullah Kayani  | 41665       | Documentation                      |
| 4 | Jam Awais Shahid | 41567       | Testing                            |

# 1. Problem Statement

Assembly language programming requires students to work directly with CPU registers, memory addressing, and low-level control flow — concepts that are abstract without a concrete, interactive application. This project addresses that gap by implementing a fully functional Number Guessing Game in x86 Assembly using the MASM assembler and Irvine32 library.

The game challenges a user to guess a randomly generated secret number within a selected difficulty range. The system provides directional hints (too high / too low), tracks all guesses in an array, validates user input, computes a score based on efficiency, and maintains a running total across multiple rounds — all implemented entirely in x86 Assembly language.

## 2. Objectives

The project was designed to demonstrate the following Assembly Language concepts:

- Implement a menu-driven interface using branching and jump instructions
- Use arrays (DWORD array) to store and display guess history per round
- Apply loops (guessLoop, mainLoop, clearLoop, histLoop) for iterative control flow
- Define and call separate procedures (PROCs) for modular code organization
- Handle user input and output using Irvine32 library routines
- Perform input validation and re-prompt on out-of-range entries
- Implement a scoring system using arithmetic instructions
- Support multiple difficulty levels affecting the random number range

## 3. Algorithm & Flowchart

### 3.1 High-Level Algorithm

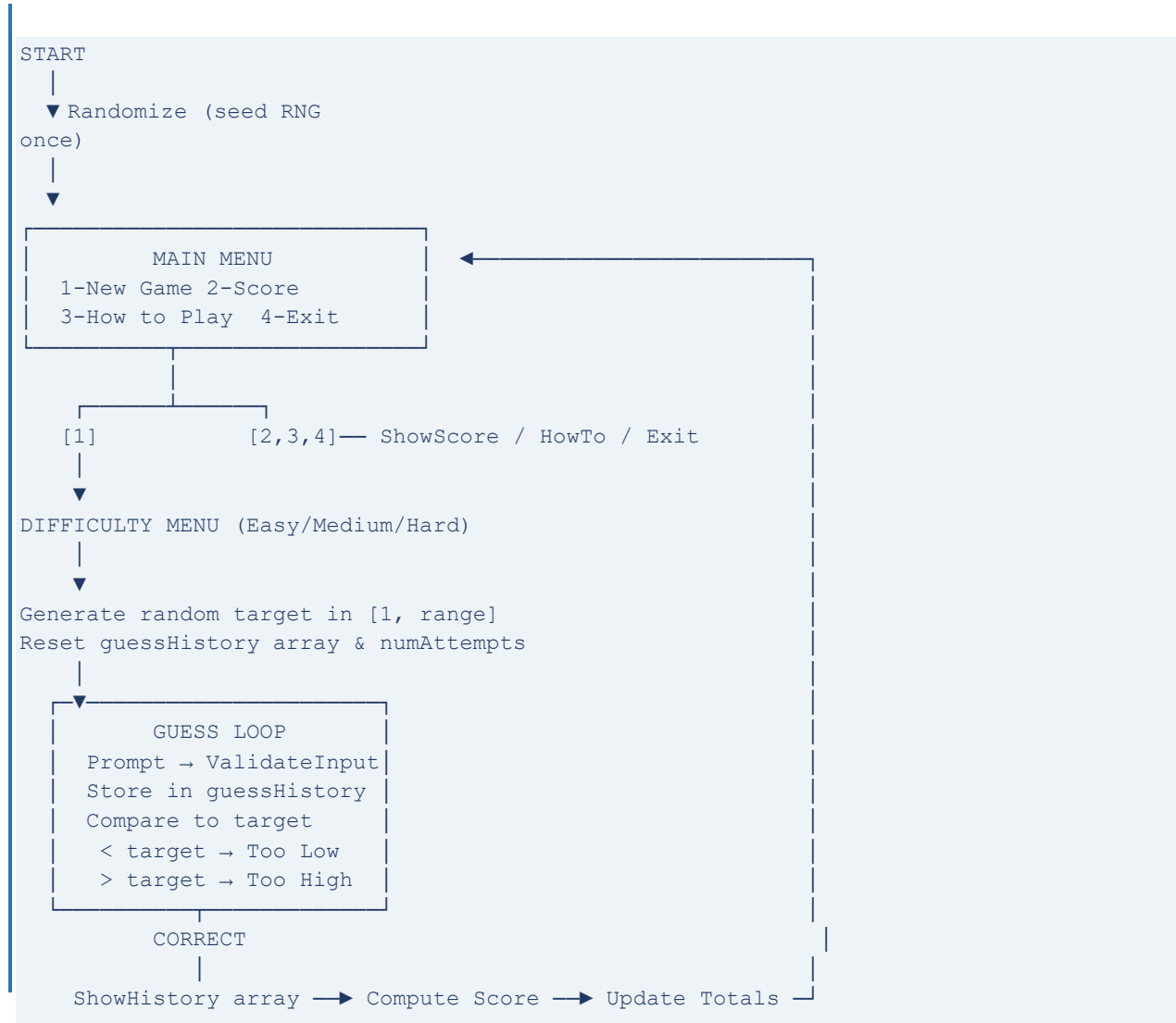
The overall program flow follows these steps:

- Step 1: Initialize random seed (Randomize), display main menu
- Step 2: Read menu choice; branch to New Game, Score, How to Play, or Exit
- Step 3 (New Game): Display difficulty menu → set range → generate random number in [1, range]
- Step 4: Enter guess loop — prompt user, validate input, store in guessHistory array

- Step 5: Compare guess to target → display Too Low / Too High → repeat until correct
- Step 6: On correct guess → display history array → compute score → update totals
- Step 7: Return to main menu; repeat until user selects Exit

## 3.2 Flowchart (Text Representation)

The following describes the program flowchart:



## 4. Implementation Details

### 4.1 Project Structure

The source code is organized into the following segments and procedures:

| Segment / PROC | Purpose                                                      |
|----------------|--------------------------------------------------------------|
| .DATA          | All variables, arrays, and string constants                  |
| Segment / PROC | Purpose                                                      |
| .CODE / main   | Entry point; runs the main menu loop with branching          |
| ShowMenu       | Clears screen and renders the 4-option main menu             |
| ShowDiffMenu   | Renders difficulty selection; sets difficulty & maxRange     |
| PlayGame       | Core game loop: RNG, guessHistory reset, guess-compare cycle |
| ValidateInput  | Re-prompts until user enters a number within [1, maxRange]   |
| ShowHistory    | Iterates guessHistory array and prints each guess            |
| ShowScore      | Displays roundsPlayed and totalScore                         |

## 4.2 Data Declarations

Key variables and the array defined in the .DATA segment:

```
targetNum    DWORD ?           ; secret number for current round
userGuess    DWORD ?           ; current input from player
numAttempts  DWORD 0           ; attempts in current round
totalScore   DWORD 0           ; cumulative score across rounds
roundsPlayed DWORD 0           ; rounds completed
difficulty   DWORD 20          ; active upper bound (default Medium)
guessHistory DWORD 10 DUP(0)   ; array: stores up to 10 guesses
```

## 4.3 Array Usage (guessHistory)

The guessHistory array is the most important data structure in the project. It is a DWORD array of 10 elements, initialized to zero at the start of each round using a loop:

```

mov ecx, MAX_GUESSES      ; loop counter = 10 lea esi,
guessHistory              ; ESI points to base of array clearLoop:
mov DWORD PTR [esi], 0    add esi, 4          ; advance
by 4 bytes (DWORD size)   loop clearLoop

```

Each guess is stored using base + offset addressing. The offset is computed as  $\text{index} \times 4$ :

```

mov ecx, numAttempts      ; index of current
guess lea esi, guessHistory ; base address mov
ebx, ecx
imul ebx, 4               ; byte offset = index *
4 add esi, ebx mov eax, userGuess
mov [esi], eax            ; guessHistory[index] = userGuess

```

## 4.4 Procedures

All major logic is separated into dedicated PROCs for clarity and reusability:

### ValidateInput

Reads an integer from the user and validates it against [1, maxRange]. If the input fails, an error message is shown and the user is re-prompted — achieved with a backward jump:

```

validateAgain:    call ReadInt
cmp eax, 1        ; below minimum?
jl badInput      mov ebx, maxRange
cmp eax, ebx      ; above maximum?
jg badInput
ret              ; EAX holds valid value
badInput:
; show error, re-prompt
jmp validateAgain

```

### Scoring Formula

Score per round is calculated as:  $\text{Score} = \max(100 - \text{attempts} \times 5, 5)$ . This rewards fewer guesses and ensures a minimum of 5 points:

```

mov eax, numAttempts
imul eax, 5          ; attempts * 5
mov ebx, 100
sub ebx, eax         ; 100 - (attempts * 5)
cmp ebx, 5 jge addScore
mov ebx, 5           ; enforce minimum score
addScore: add
totalScore, ebx

```

## 4.5 Register Usage Summary

| Register | Primary Usage in This Project                                            |
|----------|--------------------------------------------------------------------------|
| EAX      | Return value from ReadInt; arithmetic (score, random); WriteDec argument |
| EBX      | Holds targetNum for comparison; score calculation; array byte offset     |
| ECX      | Loop counter (clearLoop, histLoop); numAttempts index                    |
| EDX      | Pointer to string for WriteString (OFFSET of message)                    |
| ESI      | Base pointer for guessHistory array traversal                            |
| EIP      | Instruction pointer; manipulated by jmp, je, jl, jg, call, ret           |

## 5. Sample Input / Output

### 5.1 Main Menu

```

=====
DING DONG - NUMBER GUESSING GAME
=====

[1] New Game
[2] View Score
[3] How to Play
[4] Exit

Enter your choice (1-4): 1

```

## 5.2 Difficulty Selection

```
SELECT DIFFICULTY
[1] Easy    (1 - 10)
[2] Medium  (1 - 20)
[3] Hard    (1 - 50)
Enter difficulty (1-3): 1
```

## 5.3 Game Round (Easy Mode, target = 7)

```
Game started! Good luck!

Guess a number between 1 and 10: 5
>> Too low! Try higher.
Guess a number between 1 and 10: 9
>> Too high! Try lower.
Guess a number between 1 and 10: 7
*** Correct! You guessed it! ***
Total attempts this round: 3

--- Your Guesses This Round ---
Guess #1 : 5
Guess #2 : 9
Guess #3 : 7
```

## 5.4 Error Handling (Invalid Input)

```
Guess a number between 1 and 10: 99
[!] Input out of range. Try again.
Guess a number between 1 and 10: _
```

## 5.5 Score Board

```
=== SCORE BOARD ===
Rounds played : 2
Total score    : 165
(Score = 100 - attempts*5, min 5 per round)
```

## 5.6 Test Cases Summary

| Test Case | Input | Expected Output | Result |
|-----------|-------|-----------------|--------|
|-----------|-------|-----------------|--------|

|                           |                          |                          |      |
|---------------------------|--------------------------|--------------------------|------|
| Correct guess — 1st try   | Target=5, Guess=5        | Correct! Score=95        | PASS |
| Too low hint              | Target=8, Guess=3        | Too low! Try higher.     | PASS |
| Too high hint             | Target=4, Guess=9        | Too high! Try lower.     | PASS |
| Out-of-range input        | Easy mode, type 99       | Error msg, re-prompt     | PASS |
| Minimum score enforcement | Win in 20+ attempts      | Score = 5 (not negative) | PASS |
| Guess history display     | 3 guesses before correct | History shows 3 entries  | PASS |
| Score accumulation        | Play 2 rounds            | Total = sum of both      | PASS |
| Menu navigation           | Select 2, 3, 4 from menu | Each section loads       | PASS |

## 6. Challenges Faced

- **Array Indexing in Assembly:** Unlike high-level languages, array access requires manually computing byte offsets ( $\text{index} \times 4$  for DWORDs) and using base+offset addressing with LEA and MOV instructions.
- **Input Validation:** Creating a robust ValidateInput procedure that loops until valid input is received required careful use of backward jumps and ensuring EAX was not clobbered between the read and compare.
- **Register Preservation:** Multiple procedures use EAX, EBX, ECX, EDX simultaneously. Planning which procedure owns which register required careful design to prevent data corruption.
- **Minimum Score Logic:** Implementing  $\max(100 - \text{attempts} \times 5, 5)$  required a conditional branch after the subtraction to clamp the result, since Assembly has no built-in  $\max()$  function.
- **Random Number Range:** RandomRange returns  $[0, N-1]$  so an INC EAX was required after the call to shift the range to  $[1, N]$ . Missing this off-by-one caused incorrect boundaries during early testing.
- **Menu Loop Design:** Ensuring the main menu loop always returned correctly from all subprocedures (ShowScore, ShowHistory, etc.) required consistent use of RET and controlled JMP back to mainLoop.

## 7. Conclusion

This project successfully demonstrates a complete, interactive application built entirely in x86 Assembly Language. By implementing the Ding Dong Number Guessing Game, the team gained hands-on



experience with the core concepts of Assembly programming — including direct register manipulation, memory addressing, procedure design, loop construction, and conditional branching.

The mandatory requirements set by the course guidelines were fully met: the program features a menu-driven interface, makes use of a DWORD array for guess history, implements six separate procedures, includes input validation with error handling, and maintains a scoring system across multiple game rounds.

Through this project, the team learned that Assembly programming demands explicit management of every computational detail that high-level languages abstract away. This deepened our understanding of how CPUs execute instructions, how memory is addressed at the byte level, and how the call stack manages procedure execution — knowledge that is fundamental to computer architecture.

## 8. Source Code

The complete, commented source code is provided below. The full executable .asm file is also included in the submission ZIP.

```
TITLE Ding Dong Guess the Number Game
.686
.MODEL flat, stdcall
.STACK 4096

INCLUDE Irvine32.inc
.DATA
    targetNum
DWORD ?    userGuess
DWORD ?

    msgWelcome    BYTE "Welcome to Ding Dong Guess the Number Game!", 0Dh, 0Ah,
0    msgPrompt    BYTE "Your turn! Guess a number between 1 and 20: ", 0
msgTooLow    BYTE "Too low! Try again.", 0Dh, 0Ah, 0    msgTooHigh    BYTE "Too
high! Try again.", 0Dh, 0Ah, 0
    msgCorrect    BYTE "Congratulations! You guessed it!", 0Dh, 0Ah, 0
.CODE main PROC
call Clrscr
    call Randomize                ; Initialize random seed

    mov eax, 20
    call RandomRange                ; Generate number [0, 19]
inc eax                    ; Make it [1, 20]    mov
targetNum, eax                ; Store the target number

    ; Display welcome message
```

```
    mov edx, OFFSET msgWelcome
call WriteString
guessLoop:  mov edx, OFFSET msgPrompt    call
WriteString    call ReadInt              ;
Read user guess    mov userGuess, eax

    mov eax, userGuess
mov ebx, targetNum
cmp eax, ebx    jl
guessLow    jg
guessHigh

    ; Correct guess    mov
edx, OFFSET msgCorrect
call WriteString    jmp
endGame
guessLow:    mov edx,
OFFSET msgTooLow    call
WriteString    jmp guessLoop
guessHigh:    mov edx,
OFFSET msgTooHigh    call
WriteString    jmp guessLoop
endGame:
call WaitMsg
exit main ENDP
END main
```